

RISING BEYOND BORDERS

Document 15 — Production Rendering & Performance Architecture
Master Specification + Claude Code Implementation Prompt

Status: Implementation specification

Purpose: Establish a production rendering, performance and deployment architecture for the RBB website. The phase addresses the client-rendered SEO limitation identified in Document 14 while preserving the approved content, routing, accessibility and visual systems from Documents 01–14.

1. Phase Objective

The current site is browser-rendered. Document 14 confirmed that non-JavaScript crawlers may see only the default homepage metadata, and that redirects are currently client-side until a host is selected. This phase defines how the site should produce production-quality HTML, assets and redirects without prematurely selecting a hosting vendor or inventing production infrastructure.

2. Rendering Strategy

The preferred direction is **pre-rendering/static generation or server-side rendering for public pages**, provided it can be introduced without changing the established React architecture unnecessarily. The implementation should choose the least disruptive approach that makes each public route's meaningful content and metadata available in the initial HTML.

- Public static/content pages should be pre-rendered wherever practical.
- Dynamic Story, Project and approved Team profiles should be pre-rendered when their approved records are known at build time.
- Unknown, draft and pending dynamic records must continue to resolve to the NotFound/noindex behavior.
- Client-side hydration may enhance the page after the initial HTML is available.
- Do not add a backend solely to solve rendering if static generation can satisfy the site's current requirements.
- Do not migrate frameworks solely for this phase unless the current stack cannot meet the rendering requirements.

3. Route Rendering Matrix

Route/content	Preferred production rendering
Home / About / Work / Impact	Pre-render/static HTML
Get Involved / Donate	Pre-render/static HTML
Contact / Team / Transparency	Pre-render/static HTML
Stories directory	Pre-render; noindex if still empty
Approved Story detail	Pre-render from approved content
Projects directory	Pre-render; noindex while empty
Approved Project detail	Pre-render from approved content
Approved Team profile	Pre-render from approved content
Policies	Pre-render pending/approved state

4. Performance Budgets

Use measurable budgets rather than subjective claims such as “fast” or “lightweight.” The exact target values below are engineering targets for this phase, not claims about current production performance.

Area	Target
Initial HTML	Meaningful page content present without waiting for client JavaScript
LCP	Target ≤ 2.5 s on representative mobile conditions
INP	Target ≤ 200 ms
CLS	Target ≤ 0.1
JavaScript	Avoid shipping page-specific code that is not needed by the route
Images	Use responsive dimensions/formats and avoid oversized downloads
Fonts	Minimize blocking font requests and avoid unnecessary third-party font traffic

5. Image Optimization

- Do not load full-resolution originals when a smaller responsive asset is sufficient.
- Use explicit width/height or aspect-ratio handling to reduce layout shift.
- Use modern formats where the build/hosting stack supports them.
- Lazy-load below-the-fold images where appropriate.
- Do not lazy-load the primary hero/LCP image if doing so delays the largest contentful element.
- Use approved images only; optimization must not introduce unapproved stock imagery.
- Missing images must fail gracefully without broken layout.

6. Font Strategy

The current implementation uses Satoshi with Arial fallback and has identified Fontshare and Google Fonts as third-party requests. RBB has not yet selected the production hosting platform or decided whether fonts should remain externally hosted.

- Evaluate self-hosting approved font files as the preferred performance/privacy option if licensing permits.
- If external font hosting remains, minimize requests and ensure the loading strategy does not block meaningful content.
- Do not change the approved typography without design approval.
- Do not claim that self-hosting is required until licensing/asset availability is confirmed.

7. JavaScript & Bundle Strategy

- Keep shared application code small and avoid loading route-specific features globally.
- Code-split genuinely heavy or route-specific features.
- Do not add client libraries solely for effects that CSS can provide.

- Do not ship disabled form/payment/provider code as active production functionality.
- Remove unused dependencies where safely possible.
- Measure bundle changes rather than assuming a dependency is harmless.

8. CSS & Rendering Performance

- Keep the existing design system centralized.
- Avoid unnecessary global styles and duplicated component CSS.
- Avoid layout shifts caused by late-loaded fonts, images or components.
- Respect reduced-motion behavior already established.
- Do not sacrifice accessibility or visual consistency merely to reduce CSS size.

9. Production Redirects

Once a hosting platform is selected, the established aliases must become real HTTP redirects at the edge/server layer where the platform supports them. Client-side redirects may remain as a development fallback, but production should not depend on JavaScript to redirect search engines or users.

- Use permanent redirects for established legacy aliases.
- Redirect directly to the final canonical route.
- Avoid redirect chains.
- Do not redirect unrelated pages merely to remove old content.
- Test every redirect after deployment.

10. Caching & Static Assets

- Use immutable/versioned caching for hashed static assets.
- Allow HTML to update independently from long-lived assets.
- Do not cache personalized or future form-submission responses as public content.
- If a CDN is used, configure it without changing canonical URL behavior.
- Document cache invalidation/redeployment behavior.

11. Environment Configuration

- Keep VITE_SITE_URL empty until the production domain is confirmed.
- Do not place secrets in VITE_* variables because Vite exposes them to the browser.
- Separate development, preview and production configuration.
- Production rendering must fail safely when optional public-origin configuration is missing.
- Do not use placeholder domains in production output.

12. Hosting Architecture

No hosting provider has been selected. The implementation should therefore remain portable. The eventual host must support the chosen rendering strategy, permanent redirects, static assets, environment configuration and the site's required routing behavior.

- Do not hard-code vendor-specific APIs unless the host is approved.

- Document the deployment assumptions needed by the application.
- Verify HTTPS, compression, caching and custom-domain support before launch.
- Verify the host's cookies and third-party runtime behavior as part of the privacy/technology audit.

13. Production Error Handling

- 404 must be a real not-found response where the hosting stack permits it.
- Build failures must fail the deployment rather than silently publishing an incomplete site.
- Runtime errors should not expose stack traces or internal configuration to visitors.
- A failed optional service must not break unrelated public pages.
- Keep user-facing error states consistent with the existing design system.

14. Pre-rendering and Content Freshness

Build-time rendering means content changes require a rebuild/redeployment unless a future CMS or runtime data source is introduced. That is acceptable for the current content architecture, which uses repository-based content files.

- Approved content records become part of the next production build.
- Draft records should not be added to production content files.
- Do not introduce a CMS solely for this phase.
- Future CMS adoption can be evaluated separately if content publishing becomes operationally necessary.

15. Performance Testing

- Run production builds, not development servers, for performance measurements.
- Test representative mobile and desktop conditions.
- Measure the homepage plus representative content/detail pages.
- Check initial HTML for meaningful content and metadata.
- Measure LCP, INP and CLS.
- Inspect JavaScript bundle sizes and route-specific loading.
- Inspect image transfer sizes and font requests.
- Check that redirects happen before page rendering in production.
- Repeat tests after major performance changes.

16. Claude Code Implementation Prompt

You are implementing Document 15 – Production Rendering & Performance Architecture for the Rising Beyond Borders website.

CONTEXT

- Documents 01-14 are implemented and approved.
- The current site is browser-rendered.
- Document 14 identified that non-JavaScript crawlers may see only the default homepage metadata.
- Client-side redirects are currently used until a hosting platform is selected.
- RBB has not selected a production host or domain.
- Do not invent production infrastructure.

GOAL

Make the public site render meaningful HTML before/without client-side JavaScript where practical, improve performance, and prepare the application for production deployment without unnecessary framework migration.

RENDERING

Prefer static generation/pre-rendering for repository-backed public content.

Use SSR only if necessary.

Do not add a backend solely for rendering.

Preserve the existing React architecture where practical.

PRE-RENDER THESE TYPES

- Home
- About
- Work and four program pages
- Impact
- Get Involved and its paths
- Donate
- Contact
- Team
- Transparency
- Policies
- Approved Story/Project/Team detail pages
- 404/not-found behavior

DRAFT/PENDING

Unknown, draft and pending content must remain non-indexable and must not become public pages.

INITIAL HTML

Ensure page content, title, description and relevant metadata are available in the initial HTML for public routes.

PERFORMANCE TARGETS

These are engineering targets, not claims about current performance:

- LCP <= 2.5s
- INP <= 200ms
- CLS <= 0.1

Measure representative mobile/desktop production builds.

IMAGES

- Responsive dimensions/formats.
- Explicit sizing/aspect ratio.
- Lazy-load appropriate below-fold images.
- Do not lazy-load the primary LCP image if it delays it.
- Graceful missing-image behavior.
- Approved imagery only.

FONTS

Current typography is Satoshi with Arial fallback.

Fontshare and Google Fonts are currently external.

Evaluate whether approved font files can be self-hosted, subject to licensing.

Do not change typography without approval.

If external hosting remains, minimize blocking requests.

JAVASCRIPT

- Keep shared bundles small.
- Code-split genuinely route-specific/heavy code.
- Avoid unnecessary dependencies.
- Disabled forms/payment functionality must not become active.
- Measure bundle changes.

REDIRECTS

Prepare the existing redirect map for host-level permanent redirects.

Do not remove client fallback if needed during development.

Avoid redirect chains.
Do not choose a host.

CACHING

- Immutable caching for hashed assets.
- HTML updates independently.
- Do not cache personalized/future form responses publicly.
- Document cache invalidation assumptions.

ENVIRONMENT

- VITE_SITE_URL remains empty until RBB confirms the production domain.
- Never put secrets in VITE_* variables.
- Separate dev/preview/prod configuration.
- Never emit placeholder domains in production.

HOSTING

Keep deployment portable.
Do not use vendor-specific APIs without approval.
Document assumptions for:

- HTTPS
- compression
- caching/CDN
- redirects
- environment variables
- custom domain
- error handling

ERRORS

- Real 404 response where supported.
- Build failures fail deployment.
- No stack traces/internal configuration exposed.
- Optional services must fail safely.

CONTENT

Repository-backed approved content is suitable for build-time rendering.
Do not introduce a CMS.
Do not add draft records to production content files.

VALIDATION

1. Production build passes with no warnings.
2. Inspect generated/served HTML for meaningful page content.
3. Inspect titles/descriptions/canonical metadata without JS.
4. Test representative pages and dynamic approved records.
5. Test draft/unknown routes.
6. Test all established redirects.
7. Measure LCP/INP/CLS.
8. Inspect bundle sizes.
9. Inspect image sizes and font requests.
10. Verify no horizontal overflow.
11. Verify accessibility/focus/reduced motion.
12. Verify no console errors.
13. Verify no secrets or placeholder production domains are emitted.
14. Confirm existing 01-14 functionality remains intact.

Do not commit or stage changes unless explicitly requested.

REPORT

Return:

- rendering strategy selected and why
- files created/modified
- routes pre-rendered
- redirect behavior
- bundle/performance measurements
- image/font changes
- environment/hosting assumptions
- validation results
- decisions that still require RBB approval.

17. Approval Checklist

- ✓ Public pages have meaningful initial HTML.
- ✓ Rendering strategy does not require unnecessary framework migration.
- ✓ Approved dynamic records can be pre-rendered.

- ✓ Draft/unknown records remain unavailable/noindex.
- ✓ Performance targets are measured on production builds.
- ✓ Images are optimized without introducing unapproved imagery.
- ✓ Font strategy is documented and does not silently change brand typography.
- ✓ Route-specific JavaScript is not loaded unnecessarily.
- ✓ Production redirects are prepared for host-level implementation.
- ✓ VITE_SITE_URL remains safe until the real domain is supplied.
- ✓ No secrets are exposed in client configuration.
- ✓ Build, accessibility, responsive and console checks pass.

Important: This phase defines the production architecture but does not select a hosting vendor, production domain, analytics service or external form/payment provider. Those decisions remain separate and must be based on RBB's actual deployment and operational requirements.